Premiers programmes Python

Écrire, exécuter, observer et corriger avec Thonny

Première NSI

Langages et programmation

Objectifs

- Situer les grands domaines de la NSI et les parcours possibles après la Première.
- Mettre en œuvre une démarche d'observation, d'hypothèse, de test et d'explication.
- Écrire, enregistrer et exécuter un programme Python dans Thonny.
- Afficher un texte ou le résultat d'un calcul, puis mémoriser et modifier une valeur.
- Lire un programme, corriger une erreur simple, écrire un `if` / `else` indenté, comparer des valeurs et combiner des conditions avec `and` / `or`, écrire une boucle `while` avec une condition d'arrêt et une boucle `for` sur un intervalle connu à l'avance.

Prérequis

- Aucun prérequis en programmation.
- Savoir lire une consigne et utiliser clavier, souris et fichiers.

Sommaire

Partie	Contenu
Partie 1	Présentations et découverte de la spécialité NSI
Partie 2	Apprendre l'informatique : observer, tester, expliquer
Partie 3	Python, Thonny et le premier programme
Partie 4	Valeurs, calculs et affichage
Partie 5	Variables, lecture et correction
Partie 6	Choisir avec if
Partie 7	Recopier un programme : le nombre mystère
Partie 8	Comparer des valeurs
Partie 9	Combiner des conditions avec and et or
Partie 10	Répéter avec while
Partie 11	Parcourir avec for
Partie 12	Vrai ou faux : le type booléen
Partie 13	Définir une fonction
Partie 14	Décomposer avant de coder (méthode)

Partie 1 — Présentations et découverte de la spécialité NSI

Un premier temps pour faire connaissance

La première partie de la séquence installe le cadre de travail : l'enseignant, les élèves et la spécialité se présentent. L'enseignant est **Stanis Humez** ; son adresse de contact est `stanis.humez@ac-lille.fr`. Le tour de table permet de repérer les parcours, les habitudes et les attentes du groupe. Il n'est pas évalué : il sert à commencer à construire une manière de travailler ensemble.

Définir l'informatique

Avant de détailler le programme, le groupe propose une première définition de l'informatique. Il ne s'agit pas de trouver immédiatement une formule parfaite : chacun écrit une proposition, puis trois élèves confrontent leurs idées et conservent une définition commune accompagnée d'un exemple.

L'informatique peut être comprise comme la science et l'ensemble des techniques qui permettent de **représenter**, **traiter**, **stocker** et **transmettre** automatiquement de l'information à l'aide de machines et de programmes. Cette définition provisoire sera enrichie au fil de l'année : programmer, organiser des données, concevoir un réseau ou comprendre le Web sont autant de façons d'aborder cette même question.

> **Activité**
>
> **4 minutes, seul puis à trois. À produire :** une définition courte de l'informatique, écrite au brouillon, ainsi qu'un exemple. Proposer d'abord une formulation personnelle, la comparer à trois, puis retenir une définition commune.

Ce que l'on étudie en Première NSI

La spécialité Numérique et sciences informatiques représente **4 heures par semaine**. Elle ne se réduit pas à apprendre un langage de programmation : elle étudie les idées et les outils qui permettent de représenter, traiter, transmettre et organiser l'information.

Les thèmes étudiés se rencontrent et se croisent tout au long de l'année :

- les **données** : les organiser, les rechercher et les trier ;
- les **algorithmes et la programmation** : décrire une méthode, écrire puis mettre au point un programme ;
- le **Web** : comprendre les technologies qui permettent de construire une page interactive ;
- les **machines et les réseaux** : découvrir ce qui se passe « sous le capot », depuis les composants d'un ordinateur jusqu'aux échanges sur un réseau.

Ces notions permettent notamment d'écrire des programmes Python, de réaliser une page Web, de produire une visualisation de données ou de comprendre comment un système d'exploitation et un réseau organisent des échanges. Les projets donnent un but concret aux notions étudiées : ils seront construits progressivement, sans confondre vitesse de réalisation et compréhension.

Après la Première

Les élèves qui conservent la spécialité en Terminale la suivent **6 heures par semaine**. Ils approfondissent notamment les graphes, la récursivité, les bases de données et SQL, et réalisent des projets plus ambitieux. Une spécialité arrêtée à la fin de la Première compte dans le contrôle continu avec un **coefficient 8** ; une spécialité conservée en Terminale donne lieu à une épreuve terminale de **coefficient 16**. Ces règles font partie de la répartition actuellement publiée par le ministère pour le baccalauréat général.

Exercices

1. **À l'oral, en binôme.** Citer un objet ou un service numérique du quotidien et l'associer à un domaine de la NSI : données, programmation, Web ou réseaux.
2. **À l'écrit, sur papier.** Relier les quatre situations suivantes à ces domaines : trier une playlist, créer une page interactive, écrire un programme et se connecter au Wi-Fi.
3. **À l'oral, à trois.** Imaginer un projet réalisable cette année ; indiquer les deux domaines de la NSI qu'il mobilise et justifier ce choix en une phrase.

Partie 2 — Apprendre l'informatique : observer, tester, expliquer

Construire des modèles mentaux

Apprendre l'informatique ne consiste pas à mémoriser une grande liste de commandes. Il s'agit de construire progressivement des **modèles mentaux** : des explications suffisamment simples et fiables pour prévoir ce que fera une machine. Plus tard, une affectation, une boucle, une requête HTTP ou un réseau ne seront plus une succession de détails ; ils pourront être compris comme des objets organisés, liés à un but.

Une avance initiale en programmation donne parfois l'impression qu'un élève est « naturellement doué ». Elle correspond surtout à des modèles déjà rencontrés. Les compétences visées se construisent : elles ne constituent pas un plafond fixé à l'avance.

Une démarche commune à toute l'informatique

Face à un problème, une erreur ou un résultat surprenant, suivre ce cycle :

1. **observer** précisément ce qui se passe ;
2. formuler une **hypothèse** ;
3. imaginer le test le plus simple qui peut la confirmer ou l'infirmer ;
4. interpréter le résultat et modifier l'hypothèse si nécessaire ;
5. expliquer sa démarche à quelqu'un.

Cette démarche sert autant pour un programme qui ne fonctionne pas que pour un service réseau : « le serveur ne répond pas » devient successivement « est-il allumé ? », « a-t-il une adresse IP ? », « le port attendu répond-il ? », puis « l'application répond-elle ? ». Décomposer évite de modifier de nombreux éléments sans savoir lequel a eu un effet.

Le défi de la boîte noire

Une machine inconnue donne les résultats suivants : `3 → 10`, `5 → 26`, `8 → 65`. Il faut proposer un résultat pour l'entrée `10`, mais surtout expliquer la démarche de recherche. La règle `n² + 1` donne `101`, mais les quelques exemples ne suffisent pas à prouver que c'est la seule règle possible. Ce constat est utile : une bonne réponse en informatique est accompagnée d'un modèle, d'une hypothèse et d'un test.

Erreurs, exemples et IA

Un **bug** n'est pas un échec : c'est un désaccord entre ce qui était attendu et ce que fait réellement la machine. Le débogage commence donc par trois questions : qu'observe-t-on ? qu'attendait-on ? quel test minimal peut trancher ? Les exemples résolus et leur explication sont également utiles au début : ils permettent de construire des schémas avant de les réutiliser dans une situation nouvelle.

L'intelligence artificielle peut aider à comprendre, à relire une hypothèse ou à proposer une piste de diagnostic. Elle ne doit pas exécuter à la place de l'élève la compétence qu'il est précisément en train d'acquérir. Une bonne demande commence donc par une tentative personnelle et cherche une explication, pas une réponse à recopier.

Exercices

1. **À l'oral, en binôme.** Décrire une erreur utile rencontrée dans un contexte quotidien : ce qui était attendu, ce qui a été observé, puis le test à essayer.
2. **À l'écrit, sur papier.** Pour la boîte noire, écrire une hypothèse et un test qui pourrait l'infirmer. Expliquer pourquoi les trois exemples ne déterminent pas nécessairement une règle unique.
3. **À l'oral, à trois.** Établir un plan de diagnostic en quatre étapes pour la situation « le service ne répond pas », en allant du contrôle le plus simple au plus précis.

Partie 3 — Python, Thonny et le premier programme

Pourquoi ce nom ?

Le nom **Python** est un clin d'œil de son créateur, Guido van Rossum, à la troupe humoristique britannique des **Monty Python**. Ce n'est ni un sigle ni une référence au serpent : le langage porte un nom volontairement mémorable.

Guido van Rossum a étudié les mathématiques et l'informatique à l'université d'Amsterdam, puis a travaillé au CWI, un institut de recherche néerlandais. Il y a créé Python au début des années 1990. Il a ensuite travaillé dans plusieurs entreprises technologiques, notamment Google, Dropbox et Microsoft. Ce parcours rappelle qu'un langage de programmation n'est pas seulement une règle d'écriture : il évolue dans une communauté, dans des projets et dans des métiers.

Pourquoi apprendre Python ?

Python sert aujourd'hui à écrire des applications Web, automatiser des tâches répétitives, analyser des données, réaliser des simulations ou développer des prototypes. Il est aussi très présent dans les projets d'intelligence artificielle : il permet de préparer des données, d'expérimenter des modèles et de tester rapidement une idée. Sa popularité ne signifie pas qu'il est « meilleur » dans toutes les situations ; elle garantit en revanche une vaste communauté, beaucoup de bibliothèques et de documentation.

Il est choisi en NSI parce que son écriture est relativement lisible pour débuter, tout en étant utilisé bien au-delà du lycée. L'objectif n'est pas de réciter sa syntaxe, mais de formuler une instruction précise, d'observer son exécution et de comprendre le résultat obtenu.

Un **programme** est une suite d'instructions écrites pour résoudre une tâche. Python est un langage interprété : un logiciel appelé **interpréteur** lit et exécute les instructions.

Thonny : l'outil de travail

Thonny rassemble :

- un **éditeur** pour écrire et enregistrer le programme ;
- un bouton d'**exécution** pour demander à Python de le lire ;
- une **console** pour observer le résultat ou un message d'erreur.



Fig. 1 - Un premier programme est écrit dans l'éditeur puis son résultat est lu dans la console.

Un premier programme peut être aussi court que ceci :

```
print("Bonjour, NSI !")
```

`print` est une fonction déjà disponible dans Python. Les parenthèses contiennent ce qu'elle doit afficher. Le texte entre guillemets est une **chaîne de caractères**.

> **Activité**
>
> **5 minutes, seul. À produire :** un fichier `bonjour.py` qui affiche un message personnalisé. Créer et enregistrer le fichier dans Thonny, exécuter le programme, puis modifier le texte, sur son propre poste.

Exercices

1. **À l'écrit, sur papier.** Associer correctement les mots *Python*, *Thonny* et *programme* à « langage », « outil » ou « instructions ».
2. **À l'écrit, sur papier.** Remettre dans l'ordre les actions suivantes : écrire, enregistrer, exécuter et lire la console.
3. **Sur ordinateur, seul.** Créer `bonjour.py` avec deux appels à `print`, l'enregistrer, l'exécuter et vérifier les deux lignes affichées.

Partie 4 — Valeurs, calculs et affichage

Python peut manipuler des nombres et réaliser des calculs :

```
print(7 + 5)
print(3 * 4)
print(17 / 2)
```

Les symboles `+`, `-`, `*` et `/` sont des **opérateurs**. Une expression est évaluée avant que son résultat ne soit affiché.

Valeur	Type Python	Exemple
Entier	`int`	`12`
Texte	`str`	`"Bonjour"`
Booléen	`bool`	`True` ou `False`

> **Attention**
>
> `12` et `"12"` ne sont pas la même valeur : le premier est un nombre, le second est du texte. Python les traite différemment.

La fonction `type` permet d'observer le type d'une valeur :

```
print(type(12))
print(type("12"))
```

> **Activité**
>
> **6 minutes, seul. À produire :** un tableau « prévision / résultat / type » complété. Avant chaque exécution, prévoir ce qui apparaîtra dans la console, puis vérifier dans Thonny.

Exercices

1. **À l'écrit, sur papier.** Prévoir le résultat de `print(8 - 3)` et le justifier en une phrase.
2. **À l'écrit, sur papier.** Compléter un tableau « valeur / résultat affiché / type » pour `15`, `"15"` et `False`.
3. **Sur ordinateur, seul.** Créer `calculs.py` qui affiche `7 * 6`, puis `"7 * 6"`, puis le type de `42`. Vérifier les trois résultats dans la console.

Partie 5 — Variables, lecture et correction

Une **variable** donne un nom à une valeur afin de pouvoir la réutiliser :

```
prenom = "Ada"
annee = 2026
print("Bonjour", prenom)
print(annee + 1)
```

Le signe `=` signifie ici « affecter » : la valeur à droite est mémorisée sous le nom placé à gauche. Une variable peut ensuite recevoir une nouvelle valeur :

```
points = 12
points = points + 3
print(points)
```

Après la seconde affectation, `points` vaut `15`.

Prévoir, exécuter, expliquer

Avant de lancer un programme, prendre l'habitude de :

1. **prévoir** ce qui devrait être affiché ;
2. **exécuter** dans Thonny ;
3. **expliquer** l'écart éventuel entre la prévision et le résultat.

Un message d'erreur est une information utile. Par exemple, Python ne connaît pas `prenom` si le programme essaie de l'afficher avant de lui avoir donné une valeur : c'est une erreur de nom.

> **À retenir**
>
> Un programme est lu instruction après instruction. Une variable mémorise une valeur ; l'affectation peut la remplacer. Tester une petite modification après chaque étape aide à comprendre et à corriger.

Mélanger les types : une vraie erreur

Additionner un texte et un nombre avec `+` n'est pas autorisé en Python :

```
age = 16
print("J'ai " + age + " ans")
```

Cette instruction lève systématiquement une **erreur de type** (`TypeError`) : ce n'est pas une faute de frappe ni un cas particulier, `+` entre un `str` et un `int` échoue toujours. La correction ne change pas la valeur, elle convertit son type avec `str()` :

```
age = 16
print("J'ai " + str(age) + " ans")
```

> **Attention**
>
> `+` entre deux textes les assemble (`"7" + "6"` donne `"76"`), `+` entre deux nombres les additionne (`7 + 6` donne `13`), mais `+` entre un texte et un nombre est toujours une erreur — il n'existe pas de cas où cela fonctionne.

Exercices

1. **À l'écrit, sur papier.** Prévoir puis vérifier le résultat :

   ```
   nombre = 8
   nombre = nombre * 2
   print(nombre)
   ```

2. **Sur ordinateur, seul.** Corriger le programme suivant pour qu'il affiche `Bonjour, Ada !` :

   ```
   prenom = Ada
   print("Bonjour, " prenom "!")
   ```

3. **Sur ordinateur, seul.** Écrire un programme qui stocke votre prénom et votre année de naissance, puis affiche un message d'accueil et l'année suivante.
4. **À l'écrit, sur papier.** Prévoir la valeur finale de chaque variable, dans l'ordre :

```
a = 5
b = 2
a = a + b
b = a - b
a = a - b
print(a, b)
```

5. **À l'écrit, sur papier.** Pour chaque ligne, dire si elle s'exécute ou lève une erreur ; si elle lève une erreur, proposer la correction :

```
a = "3" + 3
b = "score : " + 12
c = str(3) + "3"
```

6. **Sur ordinateur, seul.** Corriger le programme suivant pour qu'il s'exécute sans erreur :

```
prenom = "Ada"
age = 17
print("Bonjour " + prenom + ", tu as " + age + " ans.")
```

Partie 6 — Choisir avec `if`

Un `if` permet à un programme de faire des choses différentes selon la situation. Il évalue une **expression booléenne** — une expression qui vaut `True` (vrai) ou `False` (faux) — et n'exécute le bloc qui suit que si elle est vraie :

```
if True:
    print("Ce message s'affiche.")

if False:
    print("Ce message ne s'affiche jamais.")
```

`else` définit ce qui se passe dans l'autre cas :

```
if False:
    print("Bloc du if.")
else:
    print("Bloc du else.")
```

L'indentation délimite le bloc

Ce qui est décalé (indenté) après `if ... :` ou `else :` appartient au bloc correspondant. Une ligne ramenée au même niveau que `if` n'en fait plus partie :

```
if True:
    print("Dans le bloc.")
print("Après le bloc.")
```

Une condition sur une variable

Le `if` peut évaluer une expression portant sur une variable, par exemple une comparaison :

```
age = 17
if age >= 18:
    print("Majeur")
```

```
else:
    print("Mineur")
```

`age >= 18` est l'expression booléenne évaluée par le `if` : c'est elle qui vaut `True` ou `False`, pas `age` seule.

> **À retenir**
>
> Un `if` évalue toujours une expression booléenne. Le bloc indenté qui suit ne s'exécute que si elle est vraie ; `else` définit le bloc exécuté sinon.

Exercices

1. **À l'écrit, sur papier.** Prévoir le résultat puis justifier :

   ```
   if False:
       print("X")
   else:
       print("Y")
   ```

2. **À l'écrit, sur papier.** Pour chaque version, dire si elle s'exécute normalement ou provoque une erreur d'indentation :

   ```
   # Version 1
   if True:
       print("ok")

   # Version 2
   if True:
   print("erreur ?")

   # Version 3
   if True:
       print("ok")
       print("toujours dans le bloc")
   ```

3. **Sur ordinateur, seul.** Écrire `majorite.py` : créer une variable `age`, écrire un `if` / `else` qui affiche `"Majeur"` ou `"Mineur"` selon sa valeur, puis tester avec deux valeurs différentes de `age`.

Partie 7 — Recopier un programme : le nombre mystère

Avant Internet, des magazines comme **Hebdogiciel** publiaient chaque semaine le code complet de petits jeux, à recopier à la main sur son ordinateur (par exemple en BASIC sur Thomson MO5). Recopier plusieurs pages n'apprend pas à *écrire* du code depuis rien. Mais un programme assez court, on apprend à le **lire**, à **corriger ses fautes de frappe**, et à comprendre ce qu'il fait une fois qu'il tourne enfin.

Le programme suivant est à recopier **intégralement**, caractère par caractère, sans le raccourcir ni le simplifier :

```
import random

nombre_mystere = random.randint(1, 20)
proposition = int(input("Devine un nombre entre 1 et 20 : "))

if proposition == nombre_mystere:
    print("Bravo, tu as trouvé !")
else:
    if proposition < nombre_mystere:
        print("C'est plus grand !")
```

```
    else:
        print("C'est plus petit !")
```

`random.randint` tire un nombre entier au hasard entre les deux bornes données : à accepter tel quel, sans l'expliquer en détail, exactement comme `print` en son temps.

> **À retenir**
>
> Recopier un programme entraîne la lecture et la correction, pas l'écriture. Une erreur de frappe se corrige une à la fois, en relisant à chaque fois le message de Thonny — une erreur peut en cacher une autre.

Exercice

1. **Sur ordinateur, seul.** Recopiez le programme ci-dessus dans un fichier `mystere.py`, corrigez les erreurs jusqu'à ce qu'il s'exécute sans erreur, puis testez-le plusieurs fois. Expliquez ensuite, ligne par ligne, ce que fait chaque instruction.

Partie 8 — Comparer des valeurs

Un programme doit souvent prendre une décision : un âge a-t-il atteint une limite ? un score est-il supérieur à un autre ? Pour répondre, Python **compare** deux valeurs et produit un résultat qui ne peut avoir que deux états : `True` ou `False`.

Écriture	Signification	Exemple	Résultat
`==`	est égal à	`3 == 3`	`True`
`<`	est strictement inférieur à	`2 < 5`	`True`
`>`	est strictement supérieur à	`8 > 10`	`False`
`<=`	est inférieur ou égal à	`4 <= 4`	`True`
`>=`	est supérieur ou égal à	`6 >= 9`	`False`

> **Attention**
>
> `=` et `==` n'ont pas le même rôle :
>
> ```
> score = 12       # affectation : score reçoit 12
> score == 12      # comparaison : score vaut-il 12 ?
> ```
>
> Écrire `if score = 12:` provoque une erreur de syntaxe : un `if` attend une comparaison, pas une affectation.

Une comparaison devient utile lorsqu'elle commande un `if` / `else` :

```
age = 16

if age >= 15:
    print("L'inscription est possible.")
else:
    print("L'inscription n'est pas encore possible.")
```

Exercices

1. **À l'écrit, sur papier.** Sans exécuter, indiquer `True` ou `False` pour chaque ligne : `12 == 12`, `12 < 12`, `4 <= 3`, `9 >= 9`, `"chat" == "chat"`.

2. **Sur ordinateur, seul.** Corriger le programme suivant pour qu'il affiche `Accès autorisé` à partir de `age = 18`, puis tester avec `age = 17` et `age = 18` :

```
age = 18

if age = 18:
    print("Accès autorisé")
else:
    print("Accès refusé")
```

3. **Sur ordinateur, seul.** Écrire un programme avec une variable `score` qui affiche `Réussi` si `score >= 10`, sinon `À renforcer`. Tester avec `7`, `10` et `15`.

Partie 9 — Combiner des conditions avec `and` et `or`

Certaines décisions dépendent de **plusieurs** conditions à la fois. Python combine des comparaisons avec `and` (« et ») et `or` (« ou ») :

```
age = 16
a_autorisation = True

if age >= 16 and a_autorisation:
    print("Accès autorisé")
else:
    print("Accès refusé")
```

`and` exige que **les deux** expressions soient vraies pour que le tout le soit ; `or` suffit qu'**une seule** des deux le soit.

Expression	Vrai quand…
`a and b`	`a` est vrai **et** `b` est vrai
`a or b`	`a` est vrai **ou** `b` est vrai (ou les deux)

```
jour = "samedi"

if jour == "samedi" or jour == "dimanche":
    print("C'est le week-end.")
else:
    print("C'est un jour de semaine.")
```

> **À retenir**
>
> `and` et `or` combinent deux expressions booléennes (ou plus) en une seule. Le résultat reste toujours `True` ou `False`, exactement comme une comparaison simple.

Exercices

1. **À l'écrit, sur papier.** Sans exécuter, indiquer `True` ou `False` : `3 > 1 and 3 < 10`, `3 > 1 and 3 > 10`, `3 > 10 or 3 < 10`, `3 > 10 or 3 > 20`.
2. **Sur ordinateur, seul.** Écrire un programme avec deux variables `note` et `assiduite` (booléenne). Il affiche `Admis` si `note >= 10` **et** `assiduite` valent vrai, sinon `Non admis`. Tester les quatre combinaisons possibles.
3. **À l'écrit, sur papier.** Une remise s'applique si le client est étudiant **ou** si le panier dépasse 50 €. Écrire la condition Python correspondante (variables `est_etudiant` et `montant_panier` au choix), puis vérifier son fonctionnement sur deux cas contrastés.

Partie 10 — Répéter avec `while`

Une boucle `while` répète un bloc **tant qu'**une condition reste vraie :

```
compteur = 1

while compteur <= 5:
    print(compteur)
    compteur = compteur + 1
```

Ce programme affiche `1`, `2`, `3`, `4`, `5`. À chaque tour, Python réévalue la condition ; dès qu'elle devient fausse, la boucle s'arrête.

> **Attention**
>
> Si la variable de la condition n'est jamais modifiée dans le bloc, la condition reste vraie pour toujours : c'est une **boucle infinie**. Toute boucle `while` doit contenir une instruction qui rapproche la condition de sa fin (ici, `compteur = compteur + 1`).

`while` convient bien quand on ne sait pas à l'avance combien de répétitions seront nécessaires — par exemple, répéter une saisie tant qu'elle n'est pas valide :

```
reponse = input("Le mot de passe : ")

while reponse != "python":
    reponse = input("Mot de passe incorrect, réessayez : ")

print("Accès autorisé")
```

Exercices

1. **À l'écrit, sur papier.** Prévoir ce qu'affiche ce programme, puis vérifier :

   ```
   n = 10
   while n > 0:
       print(n)
       n = n - 2
   ```

2. **Sur ordinateur, seul.** Corriger la boucle infinie suivante (elle doit afficher les nombres de 1 à 5) :

   ```
   compteur = 1
   while compteur <= 5:
       print(compteur)
   ```

3. **Sur ordinateur, seul.** Écrire un programme qui redemande un nombre tant que l'utilisateur ne saisit pas un nombre strictement positif, puis affiche `Merci !` une fois cette condition satisfaite.

Partie 11 — Parcourir avec `for`

Quand le nombre de répétitions est **connu à l'avance**, `for` avec `range` est plus direct qu'un `while` :

```
for i in range(5):
    print(i)
```

Ce programme affiche `0`, `1`, `2`, `3`, `4` : `range(5)` produit cinq valeurs, de `0` à `4`. À chaque tour, la variable `i` prend la valeur suivante.

```
for i in range(1, 6):
    print(i)
```

Avec deux bornes, `range(1, 6)` produit `1`, `2`, `3`, `4`, `5` : la première borne est incluse, la seconde exclue.

Situation	Outil
Le nombre de répétitions est connu à l'avance	`for` avec `range`
On répète tant qu'une condition reste vraie, sans savoir combien de tours	`while`

À retenir

`range` sera revu plus tard dans l'année, lorsque les listes (les tableaux en Python) seront introduites : `for` sait alors parcourir directement leurs éléments. Pour l'instant, `range` sert uniquement à répéter un nombre de fois connu.

Exercices

1. **À l'écrit, sur papier.** Prévoir la liste des valeurs affichées, puis vérifier :

   ```
   for i in range(3):
       print(i)
   ```

2. **Sur ordinateur, seul.** Écrire un programme qui affiche les nombres de 1 à 10 avec `for` et `range`, puis un second programme qui produit le même résultat avec `while`. Comparer les deux écritures.
3. **Sur ordinateur, seul.** Écrire un programme qui calcule et affiche la somme des entiers de 1 à 100 à l'aide d'une boucle `for` et d'une variable accumulateur initialisée à `0`.

Partie 12 — Vrai ou faux : le type booléen

Les comparaisons écrites depuis la partie 8 produisent toutes le même genre de résultat : une expression comme `age >= 18` ne vaut pas « peut-être », elle vaut **une seule** des deux valeurs possibles, `True` ou `False`. C'est ce résultat qu'un `if` examine pour décider quel bloc exécuter.

Ces deux valeurs forment un type à part entière, appelé `bool` :

```
print(type(3))
print(type("bonjour"))
print(type(True))
```

Ce programme affiche `<class 'int'>`, `<class 'str'>`, puis `<class 'bool'>`. `bool` est un type au même titre que `int` et `str`, avec cette particularité qu'il ne contient que deux valeurs.

Attention

`True` et `False` s'écrivent avec une majuscule et sans guillemets. `true` provoque une erreur de nom ; `"True"` est un texte de quatre caractères, pas un booléen.

Le résultat d'une comparaison n'est pas réservé au `if` : c'est une valeur ordinaire, que l'on peut stocker dans une variable et afficher.

```
note = 12
est_admis = note >= 10

if est_admis:
```

```
    print("Admis")
else:
    print("Ajourné")
```

Nommer une condition la rend lisible : `if est_admis:` se lit mieux que `if note >= 10:` lorsque la condition devient longue.

Examiner tous les cas

Les opérateurs `and` et `or`, vus en partie 9, se résument par un tableau qui énumère toutes les situations possibles.

`a`	`b`	`a and b`	`a or b`
`True`	`True`	`True`	`True`
`True`	`False`	`False`	`True`
`False`	`True`	`False`	`True`
`False`	`False`	`False`	`False`

`and` exige que les deux expressions soient vraies ; `or` se contente d'une seule.

> **À retenir**
>
> Un tableau de cas répond à une question précise : la condition se comporte-t-elle correctement dans **toutes** les situations, et pas seulement dans celle qui a servi à la tester ? C'est la démarche de mise au point d'un programme : ne pas se contenter du cas qui marche, chercher celui qui ne marche pas.

Exercices

1. **À l'écrit, sur papier, puis vérification dans Thonny.** Pour chacune de ces lignes, donner la valeur affichée — ou le message d'erreur attendu, avec sa raison :

   ```
   print(5 == 5)
   print(5 = 5)
   print("5" == 5)
   print(3 > 10)
   ```

2. **Sur papier puis Thonny, seul.** Un tarif réduit s'applique aux moins de 18 ans et aux personnes de 65 ans ou plus. Écrire la condition en Python pour une variable `age`, prédire sa valeur booléenne pour `age` valant `10`, `18`, `40`, `65` et `80`, puis vérifier chaque cas dans Thonny.

Partie 13 — Définir une fonction

Depuis la première séance, vous appelez des fonctions écrites par d'autres : `print`, `int`, `type`, `range`. Un nom suivi de parenthèses est un **appel de fonction**. Il s'agit maintenant d'en écrire une.

Le point de départ est un code qui se répète :

```
print("=========")
print(" Bienvenue ")
print("=========")

print("=========")
print("   Menu   ")
print("=========")
```

Si l'on veut changer le cadre, il faut le modifier partout — et risquer d'en oublier un endroit. Une fonction permet d'écrire ce bloc une seule fois :

```
def encadrer(texte):
    print("=========")
    print(texte)
    print("=========")
```

`def` annonce une définition, `encadrer` est le nom de la fonction, `texte` est un **paramètre** (l'information qu'on lui transmet) et le bloc indenté forme son corps.

> **Attention**
>
> Exécuter ces quatre lignes n'affiche rien. `def` range une recette dans la mémoire de Python ; elle ne se déroule que lorsqu'on l'**appelle** :
>
> ```
> encadrer(" Bienvenue ")
> encadrer("   Menu   ")
> ```
>
> Une définition, deux appels, deux résultats différents.

Retourner un résultat

```
def double(nombre):
    return nombre * 2

resultat = double(21)
print(resultat)
```

`return` renvoie la valeur à l'endroit de l'appel : `double(21)` vaut `42`, que l'on peut stocker, afficher ou réutiliser dans un calcul. C'est la différence essentielle avec `print`, qui écrit dans la console sans rien rendre.

	`print(...)`	`return ...`
Effet	Écrit dans la console	Rend une valeur à l'appelant
Réutilisable ?	Non, le texte est parti	Oui, on peut la stocker
Sans lui	On ne voit rien	La fonction renvoie `None`

```
def double_affiche(n):
    print(n * 2)

def double_retourne(n):
    return n * 2

a = double_affiche(5)
b = double_retourne(5)
print(a, b)
```

Ce programme affiche `10`, puis `None 10` : la première fonction a affiché sans rien rendre, donc `a` ne contient aucune valeur utile.

Ce qui vit dans une fonction reste dans la fonction

```
def double(nombre):
    resultat = nombre * 2
    return resultat

print(resultat)
```

La dernière ligne provoque une erreur de nom : `resultat` n'existe que pendant l'exécution de la fonction. Ce n'est pas un défaut, c'est l'intérêt d'une fonction — elle ne perturbe pas le reste du programme.

Exercices

1. **À l'écrit, sur papier, puis vérification dans Thonny.** Qu'affichent les deux dernières lignes, et pourquoi ne produisent-elles pas la même chose ?

```
def triple(n):
    return n * 3

triple(4)
print(triple(4))
```

2. **Sur ordinateur, seul.** Écrire une fonction `perimetre(longueur, largeur)` qui **retourne** le périmètre d'un rectangle, puis afficher le résultat de trois appels avec des dimensions différentes. Répondre d'abord aux quatre questions de la partie 14 (entrées, variables, structure, sorties).
3. **Sur ordinateur, seul.** Écrire une fonction `est_majeur(age)` qui retourne `True` ou `False`, puis l'utiliser dans un `if` pour afficher un message. La fonction doit contenir un `return` et non un `print` : le message s'affiche à l'extérieur.

Partie 14 — Décomposer avant de coder (méthode)

Face à un énoncé, la page blanche n'est pas un manque de connaissances : c'est un manque de **méthode** pour passer de la phrase au code. Avant d'écrire la moindre ligne, répondre par écrit à quatre questions :

1. **Entrées** : qu'est-ce que le programme doit connaître au départ ?
2. **Variables** : comment je nomme chaque information ?
3. **Structure** : que doit-il répéter, et jusqu'à quand (une seule fois, tant qu'une condition reste vraie, un nombre de fois connu) ?
4. **Sorties** : qu'est-ce qu'il doit afficher, et à quel moment ?

Exemple travaillé

Énoncé : « Écrire un programme qui redemande un nombre tant qu'il n'est pas positif. »

1. Entrées : un nombre saisi par l'utilisateur.
2. Variables : `nombre`.
3. Structure : je répète tant que `nombre` n'est pas positif → `while nombre <= 0`.
4. Sorties : un message final une fois un nombre positif obtenu.

```
nombre = int(input("Entrez un nombre : "))

while nombre <= 0:
    nombre = int(input("Ce n'est pas positif, réessayez : "))

print("Merci, vous avez saisi", nombre)
```

Chaque ligne répond à une des quatre questions : rien n'est ajouté « en plus », tout était déjà décidé sur papier.

> **À retenir**
>
> Cette méthode ne se limite pas à `while` : elle s'applique à tout énoncé de programme, y compris `for` et les fonctions.

Exercices

1. **Sur papier puis Thonny, seul.** Énoncé : écrire un programme qui demande un mot de passe tant qu'il n'est pas égal à `"python"`, puis affiche `"Accès autorisé"`. Répondre aux quatre questions sur papier avant de coder.
2. **Sur Thonny, seul.** Énoncé : écrire un programme qui demande un âge, et qui refuse (redemande) tant que l'âge saisi n'est pas compris entre 0 et 120, puis affiche l'âge validé. Répondre aux quatre questions sur un brouillon avant d'ouvrir Thonny.

Corrigés

Première NSI

Écrire, exécuter, observer et corriger avec Thonny

Langages et programmation

Partie 1

1. Toute réponse est recevable si l'association est justifiée. Par exemple, une application de musique peut mobiliser des données pour stocker les titres, des réseaux pour les diffuser et une interface Web ou applicative pour les consulter.
2. Playlist : données ; page interactive : Web ; programme : algorithmique et programmation ; Wi-Fi : machines et réseaux.
3. La réponse dépend du projet choisi. Elle doit associer deux domaines de manière cohérente et les relier à une réalisation concrète.

Partie 2

1. La réponse dépend du cas choisi ; elle doit distinguer l'observation de l'attente et proposer un test concret plutôt qu'une nouvelle supposition.
2. `n² + 1` est une hypothèse compatible avec les trois exemples et donne `101` pour 10. D'autres règles peuvent convenir aux mêmes observations : un exemple supplémentaire permettrait de les départager.
3. Un plan valable commence par vérifier que le service et sa machine sont accessibles, puis réduit progressivement le problème : adresse, réseau, port, application.

Partie 3

1. Python est un langage ; Thonny est un outil ; un programme est une suite d'instructions.
2. On écrit le programme, on l'enregistre, on l'exécute, puis on lit le résultat dans la console.
3. Une solution possible contient deux lignes de la forme `print("...")`. Les deux messages doivent apparaître à l'exécution.

Partie 4

1. `print(8 - 3)` affiche `5` : Python évalue l'expression numérique avant de l'afficher.
2. `15` est un entier (`int`), `"15"` est du texte (`str`) et `False` est un booléen (`bool`).
3. La console affiche successivement `42` , `7 * 6` et `<class 'int'>` .

Partie 5

1. Le programme affiche **16** : `nombre` vaut d'abord 8, puis reçoit le résultat de `8 * 2` .
2. Une correction possible est :

```
prenom = "Ada"
print("Bonjour,", prenom, "!")
```

3. Une réponse possible est :

```
prenom = "Ada"
annee_naissance = 2010
```

```
print("Bonjour,", prenom)
print("L'année suivante sera", annee_naissance + 1)
```

4. Après `a = a + b`, `a = 7` ; après `b = a - b`, `b = 7 - 2 = 5` ; après `a = a - b`, `a = 7 - 5 = 2`. Résultat final : `a = 2`, `b = 5`.
5. `a = "3" + 3` lève une `TypeError` (texte + nombre) ; correction : `a = "3" + str(3)` ou `a = int("3") + 3`. `b = "score : " + 12` lève aussi une `TypeError` ; correction : `b = "score : " + str(12)`. `c = str(3) + "3"` s'exécute sans erreur et donne `"33"` (deux textes assemblés).
6. Une correction possible est :

```
prenom = "Ada"
age = 17
print("Bonjour " + prenom + ", tu as " + str(age) + " ans.")
```

Partie 6

1. Le programme affiche `Y` : l'expression `False` est fausse, c'est donc le bloc du `else` qui s'exécute.
2. Version 1 : s'exécute normalement, affiche `ok`. Version 2 : erreur d'indentation (`IndentationError`), la ligne `print("erreur ?")` n'est pas indentée après `if True:`. Version 3 : s'exécute normalement, affiche `ok` puis `toujours dans le bloc` (les deux lignes sont indentées au même niveau, donc dans le bloc).
3. Une réponse possible est :

```
age = 16
if age >= 18:
    print("Majeur")
else:
    print("Mineur")
```

Partie 7

1. Le programme complet corrigé est celui du listing de la partie 7 (aucune variante attendue : l'objectif est la fidélité de la recopie). Explication ligne à ligne : `random.randint(1, 20)` tire un entier au hasard entre 1 et 20 ; `int(input(...))` lit une proposition au clavier et la convertit en entier ; le premier `if` compare la proposition au nombre mystère ; en cas d'inégalité, le second `if` (imbriqué dans le `else`) indique si la proposition est trop petite ou trop grande.

Partie 8

1. `True`, `False`, `False`, `True`, `True`.
2. L'opérateur d'affectation doit être remplacé par l'opérateur de comparaison :

```
age = 18
if age >= 18:
    print("Accès autorisé")
else:
    print("Accès refusé")
```

   Avec `age = 17`, le programme affiche `Accès refusé` ; avec `age = 18`, `Accès autorisé`.
3. Une réponse possible est :

```
score = 10
if score >= 10:
    print("Réussi")
else:
    print("À renforcer")
```

Le programme affiche `À renforcer` pour `7`, puis `Réussi` pour `10` et `15`.

Partie 9

1. `True`, `False`, `True`, `False`.
2. Une réponse possible est :

```
note = 12
assiduite = True
if note >= 10 and assiduite:
    print("Admis")
else:
    print("Non admis")
```

Les quatre combinaisons (`note` ≥ 10 ou non, `assiduite` vraie ou fausse) ne donnent `Admis` que lorsque les deux conditions sont vraies en même temps.

3. Une condition possible : `est_etudiant or montant_panier > 50`. Elle est vraie dès que l'une des deux conditions l'est, y compris si les deux le sont.

Partie 10

1. Le programme affiche `10`, `8`, `6`, `4`, `2` : `n` diminue de 2 à chaque tour jusqu'à devenir négatif ou nul.
2. Il manque l'instruction qui fait progresser `compteur` vers la fin de la condition :

```
compteur = 1
while compteur <= 5:
    print(compteur)
    compteur = compteur + 1
```

3. Une réponse possible est :

```
nombre = int(input("Entrez un nombre positif : "))
while nombre <= 0:
    nombre = int(input("Ce n'est pas positif, réessayez : "))
print("Merci !")
```

Partie 11

1. Le programme affiche `0`, `1`, `2` : `range(3)` produit trois valeurs, de `0` à `2`.
2. Deux réponses possibles :

```
for i in range(1, 11):
    print(i)
```

```
i = 1
while i <= 10:
    print(i)
    i = i + 1
```

Les deux affichent les nombres de 1 à 10 ; `for` connaît d'avance le nombre de tours, `while` doit gérer lui-même la progression et l'arrêt.

3. Une réponse possible est :

```
somme = 0
for i in range(1, 101):
    somme = somme + i
print(somme)
```

Partie 12

1. Ligne par ligne :
   - `print(5 == 5)` affiche `True` : les deux valeurs sont égales.
   - `print(5 = 5)` provoque une **erreur de syntaxe**. `=` sert à affecter une valeur à une variable ; il ne peut pas se trouver là, et `5` n'est pas un nom de variable.
   - `print("5" == 5)` affiche `False` : le texte `"5"` et l'entier `5` ne sont pas de même type, donc jamais égaux. Aucune erreur ici — la comparaison est permise, c'est son résultat qui est faux.
   - `print(3 > 10)` affiche `False`.
2. La condition est `age < 18 or age >= 65`. Le mot « et » de l'énoncé français (« aux moins de 18 ans **et** aux 65 ans ou plus ») désigne deux catégories de personnes, pas deux conditions simultanées : un même âge ne peut pas être à la fois inférieur à 18 et supérieur à 65. En Python, c'est donc `or`.

`age`	`age < 18`	`age >= 65`	Condition
`10`	`True`	`False`	`True`
`18`	`False`	`False`	`False`
`40`	`False`	`False`	`False`
`65`	`False`	`True`	`True`
`80`	`False`	`True`	`True`

Le cas `age = 18` mérite attention : `<` étant strict, un élève de 18 ans tout juste ne bénéficie pas du tarif réduit. C'est bien ce que dit l'énoncé (« moins de 18 ans »).

Partie 13

1. `triple(4)` calcule bien `12` et le retourne, mais la valeur retournée n'est ni stockée ni affichée : elle est simplement perdue, et la ligne n'affiche rien. `print(triple(4))` récupère cette valeur et l'affiche : `12`. C'est la différence entre calculer un résultat et en faire quelque chose.
2. Une réponse possible :

```
def perimetre(longueur, largeur):
    return 2 * (longueur + largeur)

print(perimetre(3, 5))
print(perimetre(10, 10))
print(perimetre(1, 7))
```

Entrées : deux dimensions. Variables : les paramètres `longueur` et `largeur`. Structure : un calcul unique, sans répétition. Sorties : le périmètre, affiché à l'extérieur de la fonction.

3. Une réponse possible :

```
def est_majeur(age):
    return age >= 18
```

```
age_saisi = int(input("Entrez un âge : "))

if est_majeur(age_saisi):
    print("Majeur")
else:
    print("Mineur")
```

`return age >= 18` suffit : la comparaison produit déjà `True` ou `False` (partie 12), il est inutile d'écrire un `if` à l'intérieur de la fonction pour retourner ensuite `True` ou `False` à la main.

Partie 14

1. Une réponse possible :

```
mot_de_passe = input("Mot de passe : ")

while mot_de_passe != "python":
    mot_de_passe = input("Incorrect, réessayez : ")

print("Accès autorisé")
```

Entrées : un mot de passe saisi. Variables : `mot_de_passe`. Structure : `while mot_de_passe != "python"`. Sorties : `"Accès autorisé"` une fois la boucle terminée.

2. Une réponse possible :

```
age = int(input("Entrez un âge : "))

while age < 0 or age > 120:
    age = int(input("Âge invalide, réessayez : "))

print("Âge validé :", age)
```

Entrées : un âge saisi. Variables : `age`. Structure : `while age < 0 or age > 120` (réinvestit `or`, vu en P09). Sorties : l'âge validé, affiché une fois la condition de répétition devenue fausse.